Engenharia de Software I - Sprint 1

Aluno: Thiago Ayolphi Liuth
Professor: Tiago Garcia de Senna Carneiro

Casos de uso e critérios de aceitação

Casos de uso:

1. Definir uma quantidade total de passos a serem simulados
2. Declarar um Sistema sozinho
3. Declarar um Fluxo sozinho
4. Adicionar um sistema de origem ao fluxo
5. Adicionar um sistema de destino ao fluxo
6. Sistema origem de múltiplos fluxos
7. Sistema destino de múltiplos fluxos
8. Múltiplos fluxos conectados como entrada e saída a diversos sistemas, formando loops (ciclos fechados)
9. Permitir definir variáveis livres que interferem em fluxos, mas que não recebem interferência;
10. Permitir definir que o valor de um sistema atue como varíavel de um fluxo

Exemplo de declaração de varíaveis

Listing 1: Declaração de variáveis

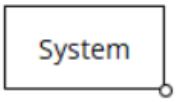
```
Simulation sim;
System system1, system2, system3;
Flow flow1, flow2, flow3;
sim.addSystem(system1);
sim.addSystem(system2);
sim.addSystem(system3);
sim.addFlow(flow1);
sim.addFlow(flow2);
sim.addFlow(flow3);
```

1) Definir número de passos

Simulation sim(10);
or
sim.defineSteps(int num);

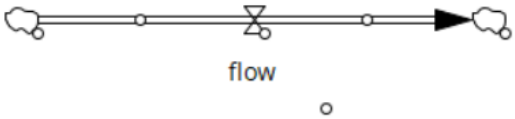
2) Declarar um Sistema sozinho

System system1; Avaliar necessidade de um construtor com um valor padrão



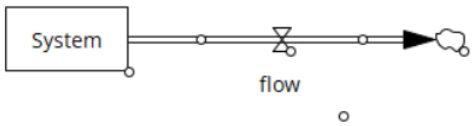
3) Declarar um Fluxo sozinho

Flow flow1; Talvez apenas a opção com origem destino seja melhor
Flow flow1(null, m)



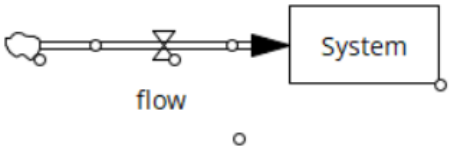
4) Adicionar sistema de origem ao fluxo

flow1.setSource(system1);



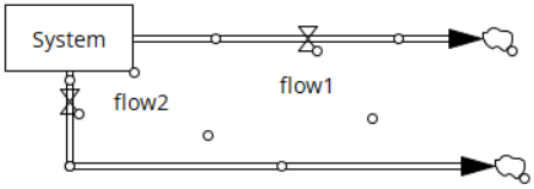
5) Adicionar um sistema de destino ao fluxo

flow1.setTarget(system1);



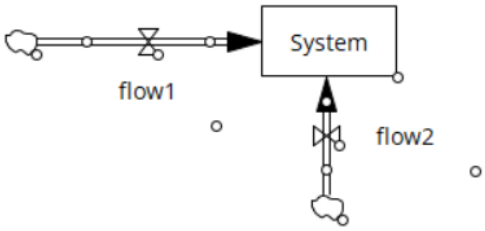
6) Sistema origem de múltiplos fluxos

flow1.setSource(system1);
flow2.setSource(system1);



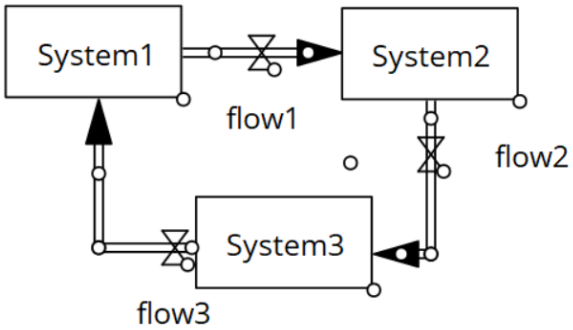
7) Sistema destino de múltiplos fluxos

```
flow1.setTarget(system1);
flow2.setTarget(system1);
```



8) Múltiplos fluxos conectados como entrada e saída a diversos sistemas, formando loops (ciclos fechados)

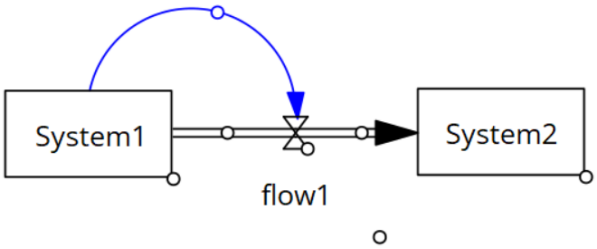
```
flow1.setSource(system1);
flow1.setTarget(system2);
flow2.setSource(system2);
flow2.setTarget(system3);
flow3.setSource(system3);
flow3.setTarget(system1);
```



9) Permitir definir variáveis livres que interferem em fluxos, mas que não recebem interferência (não são atualizadas)

10) Permitir definir que o valor de um sistema atue como variável de um fluxo

```
flow1.associateVariable(system1);
```



Atualizar sistemas a cada passo

Listing 2: Execução de steps

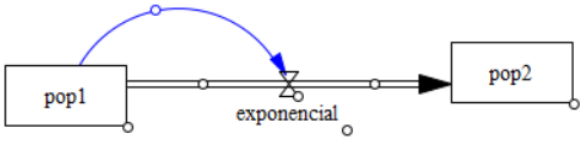
```
for(for flow in flowList){
    for(flow){
        sourceSystem = flow.getSource();
        sourceSystem.setValue(sourceSystem.getValue - flow.getValue());

        targetSystem = flow.getTarget();
        targetSystem.setValue(sourceSystem.getValue + flow.getValue());
    }
}
```

Critérios de Avaliação

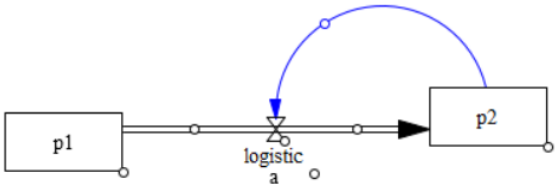
0.1 Sistema exponencial

Realiza a simulação de crescimento exponencial



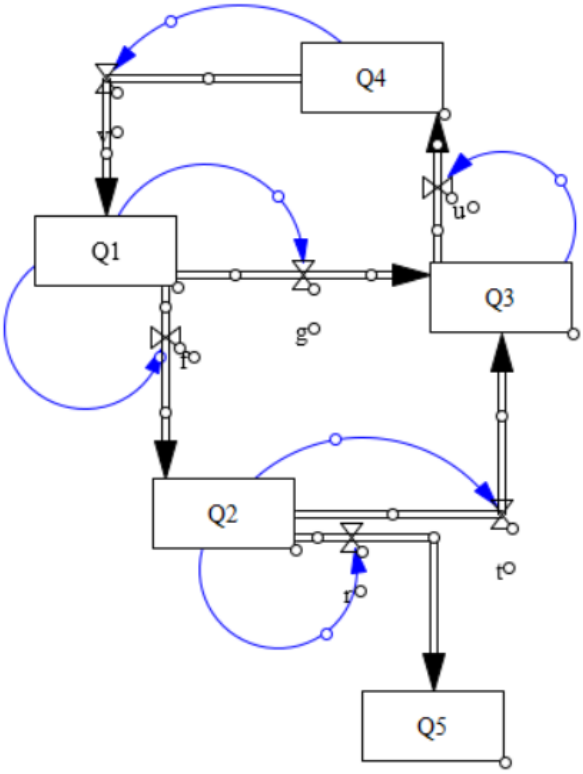
0.2 Sistema logístico

Realiza o crescimento com base em demais variáveis e condições, eventualmente atingindo um patamar estável.

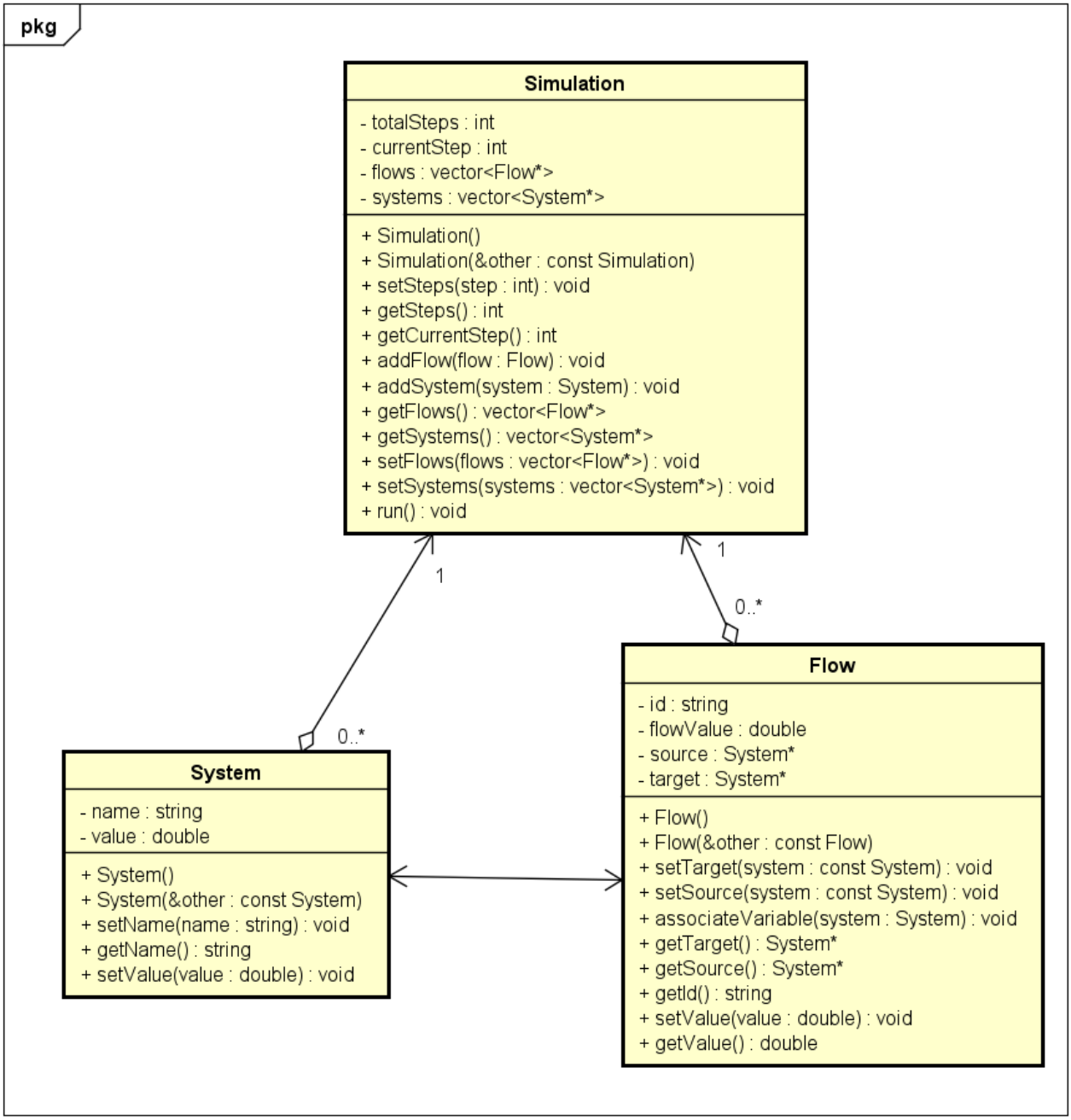


0.3 Sistema complexo

Diversos sistemas interligados por fluxos que afetam uns aos outros.


Q4
v
u
Q1
Q3
g
f
Q2
r
r
Q5

UML


pkg
Simulation
- totalSteps : int
- currentStep : int
- flows : vector<Flow*>
- systems : vector<System*>
+ Simulation()
+ Simulation(&other : const Simulation)
+ setSteps(step : int) : void
+ getSteps() : int
+ getCurrentStep() : int
+ addFlow(flow : Flow) : void
+ addSystem(system : System) : void
+ getFlows() : vector<Flow*>
+ getSystems() : vector<System*>
+ setFlows(flows : vector<Flow*>) : void
+ setSystems(systems : vector<System*>) : void
+ run() : void
1
0..*
1
0..*
System
- name : string
- value : double
+ System()
+ System(&other : const System)
+ setName(name : string) : void
+ getName() : string
+ setValue(value : double) : void
Flow
- id : string
- flowValue : double
- source : System*
- target : System*
+ Flow()
+ Flow(&other : const Flow)
+ setTarget(system : const System) : void
+ setSource(system : const System) : void
+ associateVariable(system : System) : void
+ getTarget() : System*
+ getSource() : System*
+ getId() : string
+ setValue(value : double) : void
+ getValue() : double